

Do texto ao texto

Como uma frase se transforma nos números processados por um LLM
e volta a se transformar em texto

a frase

0 limite é o cé u

os números

46 71080 4046 297 64934 84



tokenizador



decodificador

A pergunta

Uma frase entra como letras e sai como letras. No meio, só existem matrizes de números. Este seminário percorre o caminho inteiro, nos dois sentidos.

Estrutura da apresentação

1

Pergunta e motivação

Por que a tokenização decide o que o modelo consegue fazer.

2

Intuição e mapa conceitual

O caminho completo em um diagrama, sem matemática.

3

Explicação técnica

Tokens, BPE, embeddings, decoder, logits e softmax.

4

Visualização e artefato

Notebook que mostra cada etapa em uma frase real.

5

Literatura e projeto

Artigos de referência e conexão com fine-tuning.

6

Síntese e limites

O que fica de pé e o que a tokenização quebra.

1

Pergunta e motivação

O funil por onde passa tudo que um LLM lê e escreve.

- A pergunta orientadora
- Por que tokens importam
- O custo em português

A pergunta orientadora

Como uma frase se transforma nos números processados por um LLM e volta a se transformar em texto?

1 Ida

texto → números

O tokenizador quebra a frase em peças do vocabulário e troca cada peça por um inteiro. Depois cada inteiro vira um vetor.

2 Meio

números → números

O decoder mistura os vetores com atenção e devolve, para a última posição, uma pontuação para cada token possível.

3 Volta

números → texto

Softmax transforma pontuações em probabilidades, escolhemos um token e ele volta como entrada. O ciclo recomeça.

Cada uma das três etapas é uma parte da seção 3.

Por que isso não é um detalhe de implementação

O token é a unidade de conta de tudo: contexto, custo, velocidade e até os erros famosos dos LLMs.

Contexto e custo

A janela de contexto e o preço da API são medidos em tokens, nunca em palavras. O mesmo texto em português ocupa mais janela que em inglês.

O modelo não vê letras

Ele vê IDs. Por isso erra ao contar letras de uma palavra ou ao somar números longos: a informação de caractere foi embrulhada dentro do token.

Fine-tuning herda o tokenizer

Ao ajustar um modelo, o vocabulário vem congelado do pré-treino. Trocar ou estender o vocabulário exige redimensionar a matriz de embeddings.

1,32×

mais tokens em português

O mesmo parágrafo sobre modelos de linguagem, medido com o tokenizer `cl100k_base` (GPT-4): 45 tokens em português contra 34 em inglês. Na prática: menos texto cabe na janela, cada chamada custa mais e o treino em português consome mais passos para a mesma quantidade de informação.

Medição própria com tiktoken · cl100k_base

2

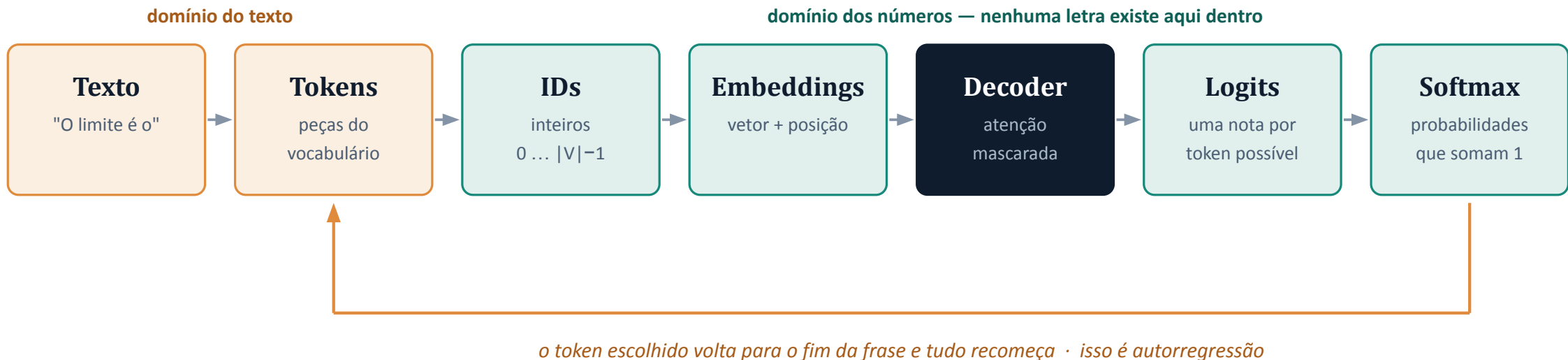
Intuição e mapa conceitual

O caminho inteiro em um diagrama, antes da matemática.

- O pipeline completo
- Três intuições que bastam

O mapa: onde cada conceito entra

Sete estágios. Os quatro do meio são puro cálculo numérico — o texto só existe nas pontas.



Leia assim:

o tokenizador e o softmax são as duas portas entre linguagem e álgebra. Entre elas, um LLM é apenas multiplicação de matrizes: nenhuma etapa do meio sabe o que é uma letra, uma sílaba ou uma palavra.

Três intuições que sustentam o resto

1

Texto vira uma lista de peças

As peças não são palavras. São pedaços frequentes aprendidos a partir de um corpus — às vezes uma palavra inteira, às vezes duas letras.

"limite" → uma peça

"céu" → duas peças

2

Cada peça vira uma direção

O ID sozinho não tem significado: 71080 não é maior nem melhor que 46. O significado está no vetor que o modelo aprendeu para aquele ID.

ID → vetor de 768 números

sentido $\approx$ direção no espaço

3

O modelo só faz uma coisa

Ele recebe um prefixo e devolve uma distribuição de probabilidade sobre o próximo token. Resumir, traduzir e responder são tudo isso repetido.

entrada: prefixo

saída: $P(\text{próximo token})$

3

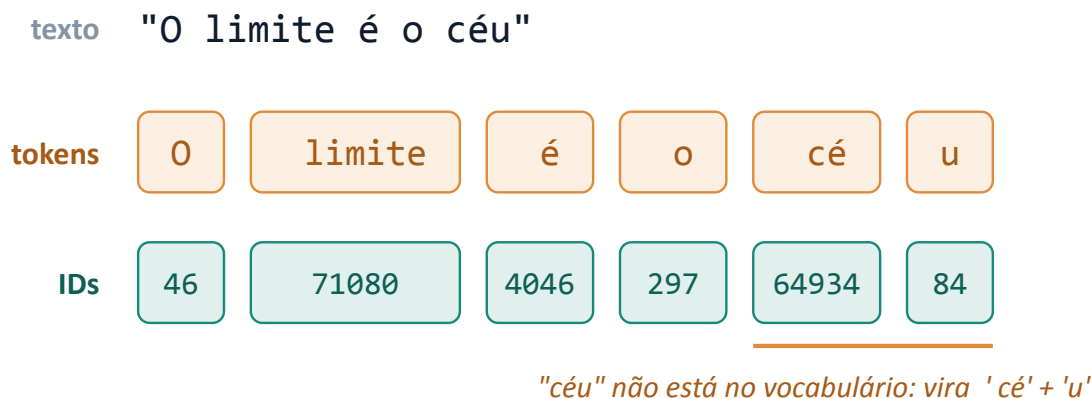
Explicação técnica

Abrindo cada caixa do pipeline, com exemplos medidos.

- Tokens, IDs e vocabulário
- Tokens especiais
- BPE e treino do tokenizer
- Português × inglês
- Embeddings de token e posição
- Transformer decoder
- Logits, softmax e previsão

Tokens, IDs e vocabulário

A mesma frase da capa, agora com nomes técnicos em cada camada.



trecho do vocabulário

71078 ' PREFIX '

71079 ',**'

71080 ' limite '

71081 ' _heat '

71082 '%, '

IDs vizinhos não têm relação: é lista, não escala.

Token

A menor peça que o modelo manipula: uma palavra, um pedaço de palavra ou um sinal de pontuação. O espaço entra junto com a palavra seguinte.

ID

A posição daquele token na lista do vocabulário. É só um índice: não há ordem nem grandeza. 71080 e 46 são igualmente arbitrários.

Vocabulário

A lista fixa de todos os tokens possíveis, definida ao treinar o tokenizer.

Tokens especiais: a pontuação que o modelo entende

Entradas do vocabulário que não vêm do texto do usuário. Elas marcam fronteiras, preenchem lotes e organizam o diálogo.

< endoftext >	Fim de documento. Também sinaliza ao gerador que pode parar.	<i>GPT-2, GPT-4</i>	[CLS] [SEP]	Início da sequência e separador entre dois trechos.	<i>BERT</i>
[PAD]	Enche sequências curtas para o lote virar uma matriz. A máscara manda ignorá-lo.	<i>todos</i>	[UNK]	Token desconhecido. O BPE em nível de byte o tornou desnecessário.	<i>legado</i>
[MASK]	Posição escondida que o modelo deve adivinhar durante o pré-treino.	<i>BERT</i>	< im_start >	Abre um turno de conversa com o papel: system, user ou assistant.	<i>modelos de chat</i>

Por que isso importa para fine-tuning

Ao ajustar um modelo de chat, o texto de treino precisa ser formatado com exatamente os mesmos tokens especiais do pré-treino. Um <|im_start|> trocado por texto literal faz o modelo aprender a imitar a marcação em vez de obedecê-la — é uma das causas mais comuns de fine-tuning que "não pega". Use sempre `tokenizer.apply_chat_template`.

BPE: o algoritmo, rodado à mão

Byte Pair Encoding começa com caracteres soltos e funde, repetidamente, o par vizinho mais frequente do corpus.

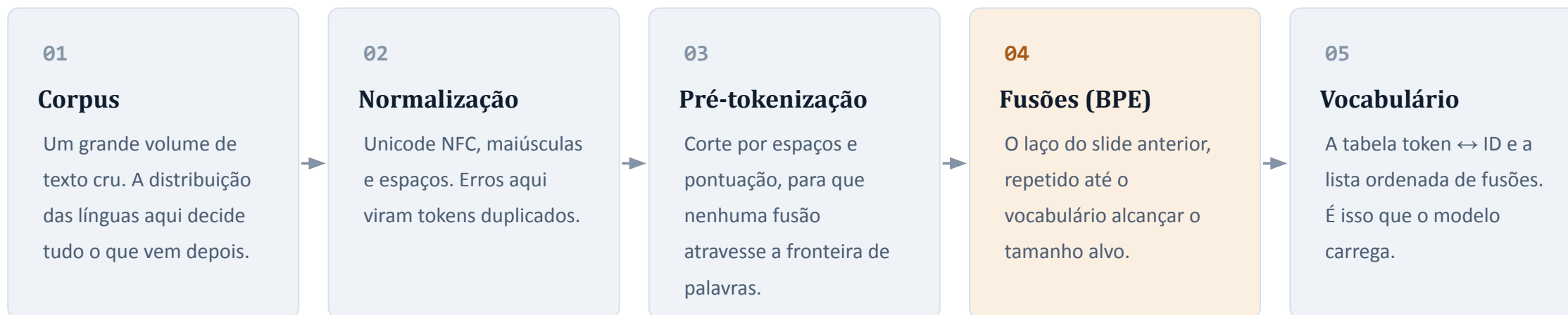
Corpus de brinquedo		Cinco primeiras fusões aprendidas			
baixa	×5	passo 1	ai	freq 16	b ai x a · c ai x ote
baixo	×3	passo 2	aix	freq 16	b aix a · c aix ote
caixa	×4	passo 3	baix	freq 10	baix a · baix inho
caixote	×2	passo 4	a</w>	freq 9	baix a</w> · caix a</w>
baixinho	×2	passo 5	caix	freq 6	caix a</w> · caix ote

O que aconteceu no passo 3

Sem gramática e sem dicionário, contando apenas frequência, o algoritmo isolou o radical `baix-` de "baixa", "baixo" e "baixinho". É esse tipo de regularidade morfológica que faz um vocabulário de subpalavras funcionar.

- Nada aqui é escrito à mão: as fusões são aprendidas do corpus.
- A ordem das fusões é o modelo — aplicá-las na mesma ordem retokeniza qualquer texto novo.
- `</w>` marca fim de palavra e impede fundir através do espaço.
- Parar cedo → peças curtas; parar tarde → vocabulário grande e raro.

Treinar um tokenizer: o que se escolhe e o que se aprende



As duas decisões que mais doem depois

Tamanho do vocabulário $|V|$

Vocabulário grande encurta as sequências (menos passos de atenção) mas engorda a matriz de embeddings e a camada de saída, ambas proporcionais a $|V|$. Modelos atuais ficam entre 32 mil e 256 mil.

Composição do corpus

Se o corpus do tokenizer é quase todo em inglês, as fusões úteis para o português nunca são aprendidas — e cada palavra nossa vira três ou quatro pedaços.

Byte-level BPE (GPT-2 em diante): as unidades iniciais são os 256 bytes, não os caracteres. Qualquer texto — emoji, chinês, código — é sempre representável, e o token [UNK] deixa de existir.

O imposto linguístico da tokenização

Um mesmo conteúdo custa mais caro — em dinheiro, em contexto e em qualidade — dependendo do idioma em que foi escrito.

1

A ilusão da igualdade

As fusões do tokenizador são aprendidas num corpus dominado pelo inglês. O resultado é uma taxa de fragmentação desproporcional entre idiomas e entre alfabetos.

2

O custo da sub-representação

As APIs cobram por token processado e gerado. Como a mesma informação exige mais tokens fora do inglês, o falante paga mais caro pelo mesmo conteúdo.

3

A perda de desempenho

Texto fragmentado ocupa mais janela de contexto. Sobra menos espaço para instruções e exemplos — justamente o que alimenta o aprendizado pelo prompt.

o mesmo conceito, dois idiomas

português

des

env

olvimento

3 tokens

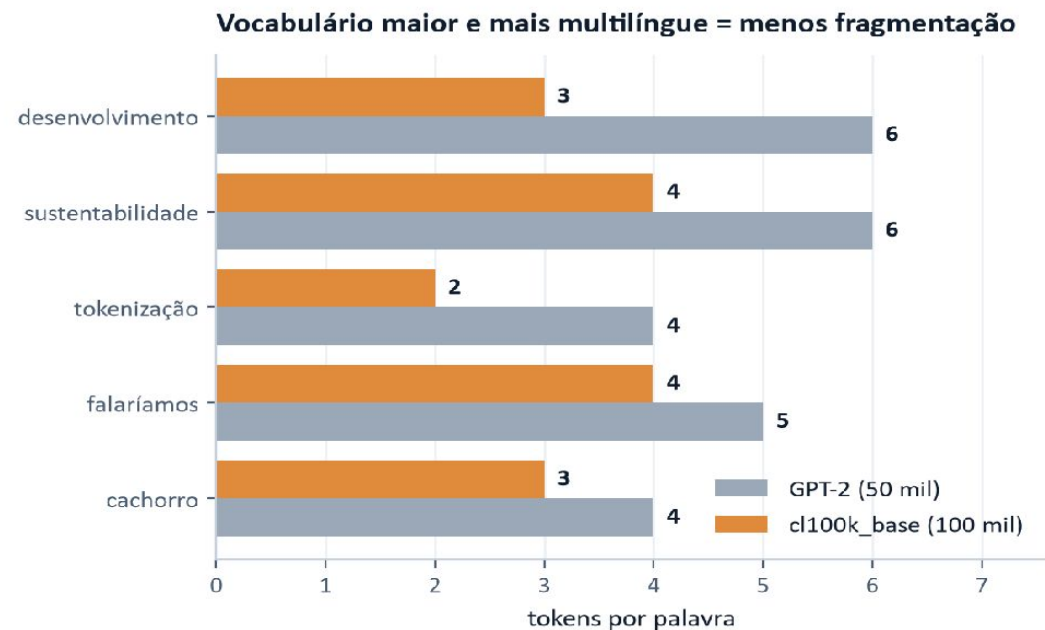
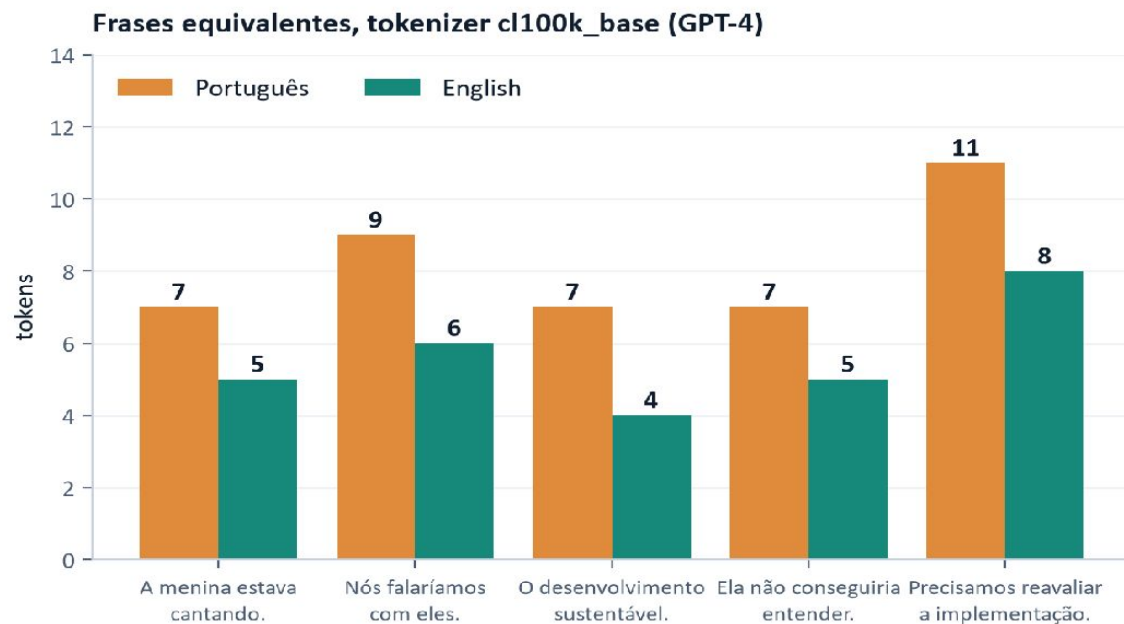
english

development

1 token

Três peças contra uma para a mesma ideia.
Multiplicado por um texto inteiro, vira fatura maior, menos contexto útil e resposta pior.

Português × inglês: o mesmo texto, mais tokens



Morfologia flexional

"falaríamos" carrega pessoa, número, tempo e modo numa palavra só. O tokenizer corta em f · alar · í · amos.

Acentos custam bytes

Em UTF-8, á, ç e ã ocupam dois bytes. No BPE em nível de byte isso vira fronteira extra de fusão.

Corpus enviesado

As fusões foram aprendidas em texto majoritariamente inglês: os padrões do português não viraram tokens.

Token embeddings: do índice ao significado

O ID sozinho não quer dizer nada. O significado mora no vetor que o modelo aprendeu para aquele ID.

Embeddings: proximidade e direção carregam significado



Projeção didática em 2D — o espaço real tem centenas de dimensões.

1 Superando índices isolados

Em vez de tratar palavras como unidades atômicas e sem relação entre si, uma rede neural aprende representações vetoriais contínuas a partir de conjuntos de dados gigantescos.

2 Semântica no espaço vetorial

Cada token ganha uma posição num mapa multidimensional. A expectativa é que palavras de significado parecido acabem próximas umas das outras nesse espaço.

3 Álgebra de palavras

O mapeamento é preciso a ponto de preservar regularidades lineares, o que permite fazer aritmética com os vetores.

```
vetor("rei") - vetor("homem") + vetor("mulher")
  ≈  vetor("rainha")
```

o exemplo clássico, de Mikolov et al. (2013): $king - man + woman \approx queen$

Embeddings de posição: o carimbo que dá ordem à frase

Sem posição, para a atenção estas duas frases são a mesma coisa:

"O cachorro mordeu o homem"

≠

"O homem mordeu o cachorro"

1 O desafio do paralelismo

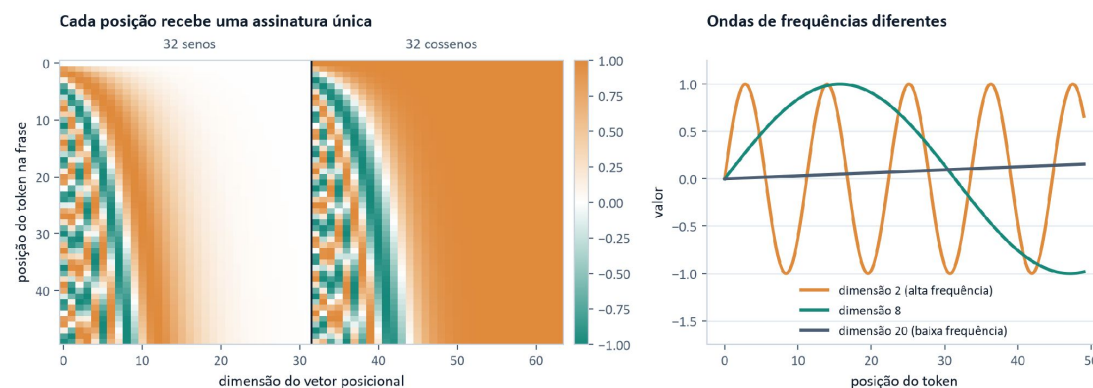
As redes antigas liam palavra por palavra; o Transformer processa todos os tokens de uma vez. Ganha velocidade, mas perde a ordem: atenção é soma, e soma não tem sequência.

2 A injeção de posição

A solução é carimbar cada token com o lugar que ele ocupa, antes de entrar na rede. O carimbo não substitui o embedding do token: é somado a ele.

3 A matemática da ordem

O carimbo é um vetor feito de ondas de seno e cosseno em frequências diferentes. Cada posição ganha uma combinação única, como os ponteiros de um relógio.

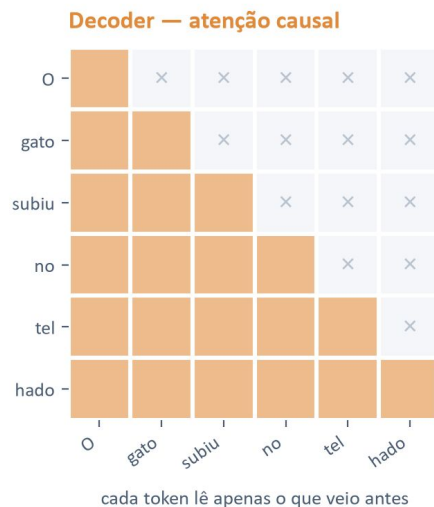
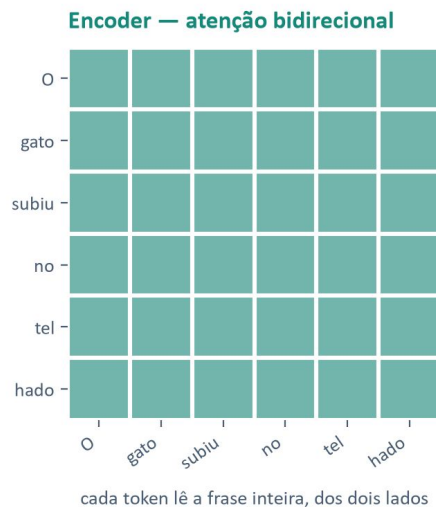


$$\text{entrada} = E[\text{id}] + P[\text{posição}]$$

Cada posição vira uma assinatura própria no mapa de calor acima, montada pelas ondas ao lado. Esse vetor é somado ao embedding do token — significado e lugar entram juntos na rede.

Encoder e decoder: as duas metades do Transformer

O artigo de 2017 propôs duas pilhas de blocos. A diferença entre elas cabe numa frase: quem cada token pode enxergar.



Encoder — lê

Recebe a sequência inteira de uma vez e devolve um vetor por token, já com o contexto dos dois lados. Não escreve: entrega representações. O pré-treino esconde palavras ao acaso e pede que sejam adivinhadas.

Decoder — escreve

A mesma pilha de blocos, com uma máscara que apaga o futuro. Cada posição enxerga só o que veio antes — e é isso que permite treinar a previsão do próximo token em todas as posições ao mesmo tempo.

Encoder-only

BERT • RoBERTa • E5

Classificação, busca semântica e embeddings de frase. Entende, mas não escreve.

Decoder-only

GPT • Llama • Mistral • Qwen

Geração e conversação. A família de todos os LLMs atuais — e o próximo slide.

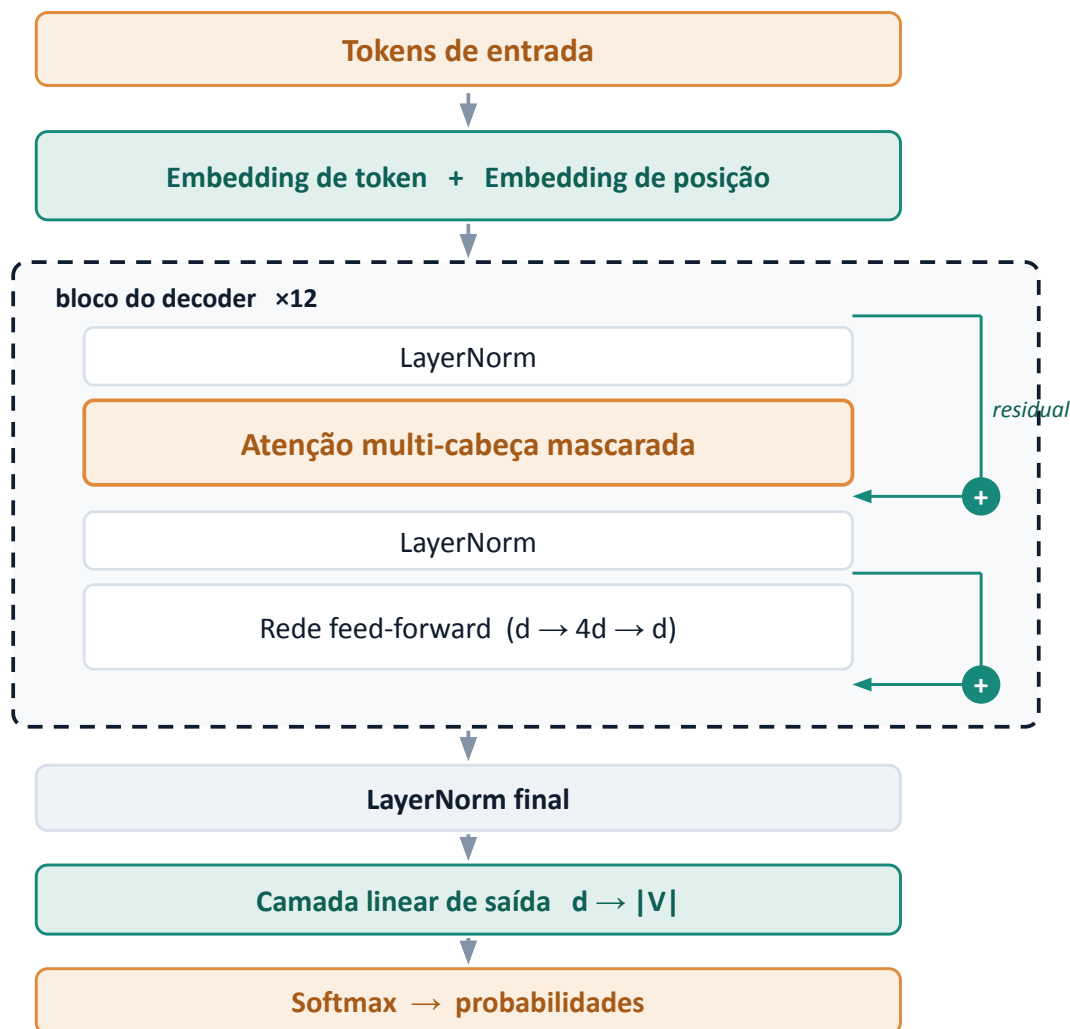
Encoder-decoder

T5 • BART • Whisper

Tradução, resumo e transcrição. O decoder consulta o encoder por atenção cruzada.

Por que os LLMs largaram o encoder: qualquer tarefa vira "continue este texto" — e uma pilha só, com um objetivo de treino só, escala melhor.

Visão geral de um Transformer decoder



Decoder-only: a família GPT, Llama e Mistral usa só esta metade do Transformer original — não há bloco encoder nem atenção cruzada.

Atenção mascarada

Cada posição olha para si mesma e para as anteriores, nunca para as seguintes. É o que permite treinar todas as posições de uma vez sem vaziar a resposta.

Feed-forward

Uma MLP aplicada a cada posição isoladamente, com a camada interna quatro vezes maior. Guarda a maior parte dos parâmetros do modelo.

Residual + LayerNorm

O atalho soma a entrada à saída de cada sub-camada. Sem ele, o gradiente não atravessa dezenas de blocos empilhados.

Camada de saída

Projeta o vetor final de d dimensões para $|V|$ pontuações — uma por token do vocabulário. Costuma reaproveitar os pesos da matriz de embeddings.

GPT-2 small: 12 blocos · 12 cabeças · $d = 768$ · 117 M parâmetros

Logits: a nota bruta de cada token do vocabulário



Não são probabilidades

Um logit pode ser $-4,2$ ou $+11,7$. Não estão entre 0 e 1 e não somam 1. Sozinho, um logit não diz nada: só a comparação com os outros tem sentido.

Só a última posição importa

Na geração, o modelo calcula logits para todas as posições, mas apenas a linha do último token é usada. No treino, todas são usadas de uma vez — daí a eficiência.

Pesos amarrados

W_u costuma ser a transposta da matriz de embeddings. O mesmo parâmetro traduz ID→vetor na entrada e vetor→nota na saída, economizando 38 M de pesos.

Exemplo

para o prefixo "O limite é o" :

céu

+6,2

chão

+4,1

mar

+3,6

teto

+1,4

livro

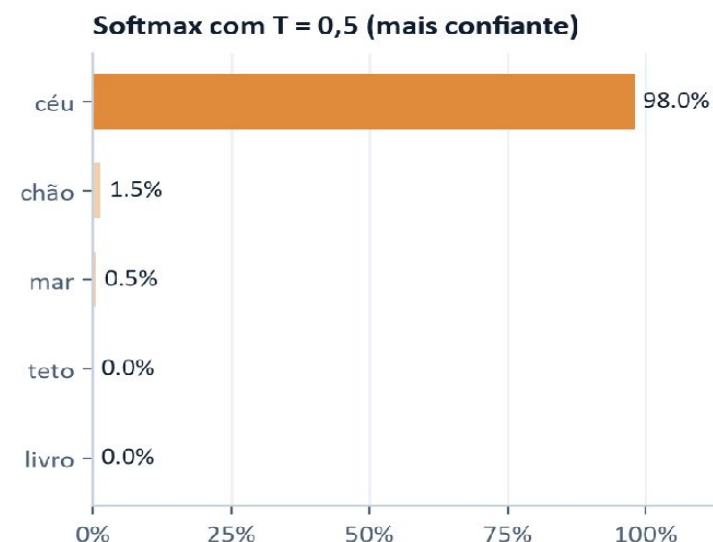
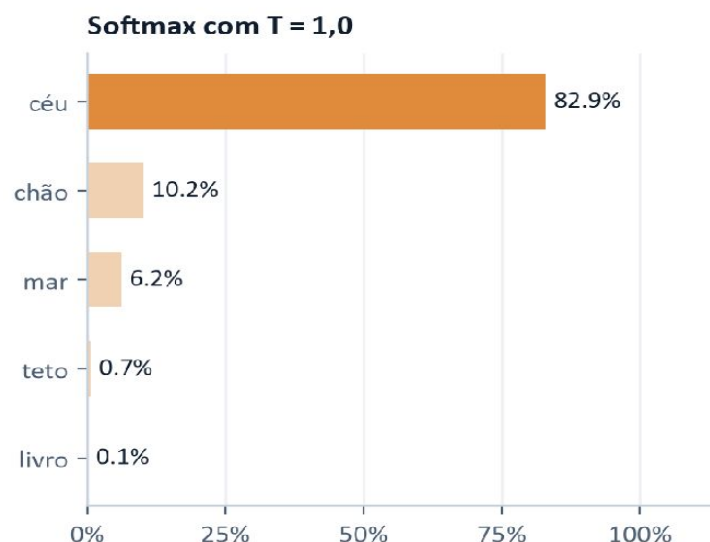
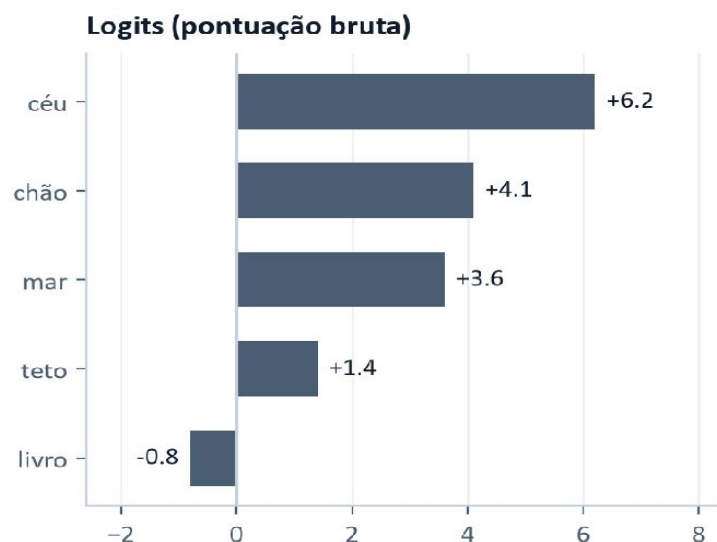
-0,8

valores ilustrativos

Softmax: de notas brutas a uma distribuição

$$P(\text{token } i) = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

exponencia para eliminar negativos · divide pela soma para totalizar 1 · T é a temperatura



Preserva a ordem

O maior logit continua sendo o mais provável.
Softmax não escolhe nada — apenas reescala.

Amplifica diferenças

A exponencial é implacável: +2,1 de vantagem em logit virou 83% contra 10% de probabilidade.

Temperatura controla o risco

$T < 1$ concentra a massa no favorito; $T > 1$ achata a distribuição e deixa o texto mais variado.

Escolher o próximo token — e recomeçar

A distribuição está pronta. Falta a política de amostragem: qual token efetivamente sai.

Gulosa

`argmax`

Sempre o mais provável.
Determinístico e seguro, mas repetitivo em textos longos.

Amostragem

`sample`

Sorteia seguindo a distribuição. Variado, porém pode escolher uma cauda absurda.

Top-k

`k = 50`

Sorteia apenas entre os k melhores. Corta a cauda com um número fixo.

Top-p

`p = 0,9`

Sorteia no menor conjunto que acumula 90% da massa. O corte se adapta à confiança do modelo.

O laço autorregressivo



O token gerado é anexado ao prefixo e tudo é recalculado. Para quando sai o token de fim — ou quando o limite de comprimento estoura.

4

Literatura e projeto

De onde vem cada ideia e como ela toca o nosso trabalho.

- Artigos fundadores
- Conexão com fine-tuning

Literatura de referência

Vaswani et al. (2017) Attention Is All You Need A arquitetura Transformer, a atenção multi-cabeça e a posição senoidal.	arquitetura Sennrich, Haddow & Birch (2016) Neural Machine Translation of Rare Words with Subword Units Traz o BPE da compressão de dados para a tokenização de texto.
Kudo & Richardson (2018) SentencePiece Tokenização sem depender de espaços, que é o que viabiliza modelos multilíngues.	tokenizers Radford et al. (2019) Language Models are Unsupervised Multitask Learners GPT-2: decoder-only, BPE em nível de byte e o fim do token [UNK].
Su et al. (2021) RoFormer: Enhanced Transformer with Rotary Position Embedding RoPE, a codificação posicional usada hoje em Llama, Mistral e Qwen.	posição Rust et al. (2021) How Good is Your Tokenizer? Mede o custo de usar um tokenizer inadequado para línguas que não o inglês.

Documentação prática

Hugging Face — curso de NLP, cap. 6 (tokenizers) · huggingface.co/docs/tokenizers · OpenAI tiktoken · Jay Alammar, The Illustrated Transformer · Karpathy, Let's build the GPT Tokenizer

Nota de método

Os números de tokenização deste seminário (contagens PT × EN, IDs da frase de exemplo, comparação GPT-2 × cl100k_base) foram medidos por nós com a biblioteca tiktoken. O traço de BPE do slide do algoritmo saiu de uma implementação própria rodando no corpus mostrado.

5

Síntese e limites

A resposta fechada e as trincas que ela deixa à mostra.

- A resposta em seis passos
- O que a tokenização quebra

A resposta, em seis passos

Como uma frase se transforma nos números processados por um LLM e volta a se transformar em texto?

01 Tokenizar

O BPE recorta a frase em peças do vocabulário — peças que ele aprendeu contando pares frequentes num corpus.

02 Indexar

Cada peça vira o seu índice na tabela. A frase agora é uma lista de inteiros, e nada mais.

03 Vetorizar

Cada índice puxa uma linha da matriz de embeddings e soma-se a ela o vetor da posição.

04 Processar

Doze blocos de atenção mascarada e feed-forward reescrevem cada vetor conforme o contexto anterior.

05 Pontuar

A camada linear final transforma o último vetor em uma nota para cada um dos 100 mil tokens.

06 Escolher

Softmax vira probabilidade, a política de amostragem escolhe um token — e ele volta ao passo 2.

Limites e próximos passos

O que a tokenização quebra

Contar letras e soletrar

"Quantos R tem morango?" é difícil porque o modelo nunca viu as letras: viu dois ou três tokens opacos.

Aritmética com números longos

Os dígitos são agrupados de forma irregular pelo tokenizer, então a mesma quantidade tem recortes diferentes.

Desigualdade entre línguas

Mais tokens para a mesma ideia significa menos contexto útil, mais custo e pior desempenho em português.

Janela e custo quadrático

A atenção compara todos os pares de posições: dobrar o comprimento quadruplica o trabalho.

Para onde a área está indo

Tokenizers multilíngues melhores

Vocabulários equilibrados por língua reduzem a penalidade do português — é o caminho mais barato para ganhar desempenho.

Modelos em nível de byte

Propostas como o MegaByte e o BLT eliminam o tokenizer e trabalham direto sobre bytes, resolvendo os erros de soletração.

Janelas longas e atenção eficiente

Variantes esparsas, agrupadas ou lineares atacam o custo quadrático e esticam o contexto.

Nosso próximo passo

Seguir do tokenizer para os pesos: como o pré-treino e o fine-tuning ajustam justamente as matrizes que vimos aqui.

Obrigado

Perguntas?

o limite é o cé u

seis tokens, seis IDs, 100 277 possibilidades a cada passo

Equipe · Treinamento e fine-tuning

Gabriel, Jolie e Mateus

Materiais: notebook, figuras e medições
disponíveis no repositório da equipe.